

UCS503P - Software Engineering Project

EduOS: A Linux-Based Academic Computing Platform

Mid-Semester Prototype Report

Submitted by:

1024170276 Dhawal Singh Jamwal

1024170275 Aaditya Kumar Yadav

1024170283 Harshit

Bachelor of Engineering - Computer Science and Engineering

Submitted to:

Dr. Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	3
1.2	Project Objectives	4
2	Methodology and Design	5
2.1	System Architecture	5
2.1.1	High-Level Design	5
2.1.2	Technical Stack	6
2.2	Implementation Details	7
2.2.1	Module 1: Arch Linux ISO Setup	7
2.2.2	Module 2: User Base and Student Environment Setup	8
2.3	System Diagrams	10
2.3.1	Use Case Diagram	10
2.3.2	Use Case Specifications	10
2.3.3	Class Diagram	11
2.3.4	Activity Diagram	12
2.3.5	Data Flow Diagrams	13
2.3.6	Sequence Diagram	16
2.3.7	Collaboration Diagram	17
2.3.8	Gantt Chart	18
3	Results and Evaluation	19
3.1	Testing Methodology	19
3.2	Performance Metrics	19
3.3	Analysis	20
4	Conclusion	22
4.1	Summary of Work	22
4.2	Key Findings	22
4.3	Future Work and Recommendations	23
4.4	Final Remarks	23

Introduction

1.1 Background and Context

University computing laboratories are required to support a wide variety of academic activities including programming, operating systems, networking, databases, and software development.

Although laboratory computers may contain similar hardware, their software environments can become inconsistent because of package changes, configuration differences, dependency problems, or changes made by previous users.

This creates difficulties for both students and laboratory administrators. Students may encounter systems that behave differently from one another, while administrators must repeatedly install, configure, repair, and maintain software across multiple machines.

EduOS is proposed as an academic computing platform in which the operating environment is controlled and reproducible. Instead of treating every laboratory machine as an independently configured computer, EduOS aims to provide a common managed academic environment.

The current mid-semester prototype focuses on the client-side foundation of EduOS. A custom Arch Linux image has been created together with a multi-user student environment and a personalized student application.

1.1.1 Problem Statement

Traditional university laboratories commonly face problems such as:

- Differences in installed software between machines.
- Missing or incompatible development tools.
- Configuration changes caused by previous users.
- Repeated manual setup and maintenance.
- Lack of a common personalized academic environment.
- Difficulty in associating a student's authenticated operating-system session with academic tasks.

EduOS attempts to address these problems by providing a controlled Linux-based academic environment that can be reproduced and extended across laboratory machines.

1.2 Project Objectives

The major objectives of EduOS are:

1. Develop a bootable customized Linux environment suitable for academic laboratory use.
2. Provide separate Linux identities and home directories for individual students.
3. Automatically configure required student accounts during system startup.
4. Provide a personalized student interface based on the authenticated Linux identity.
5. Maintain a reproducible system configuration through a version-controlled ArchISO profile.
6. Design the platform so that centralized management, laboratory profiles, task distribution, submissions, and automated evaluation can be integrated in later stages.

Methodology and Design

2.1 System Architecture

EduOS is designed as a modular academic computing platform.

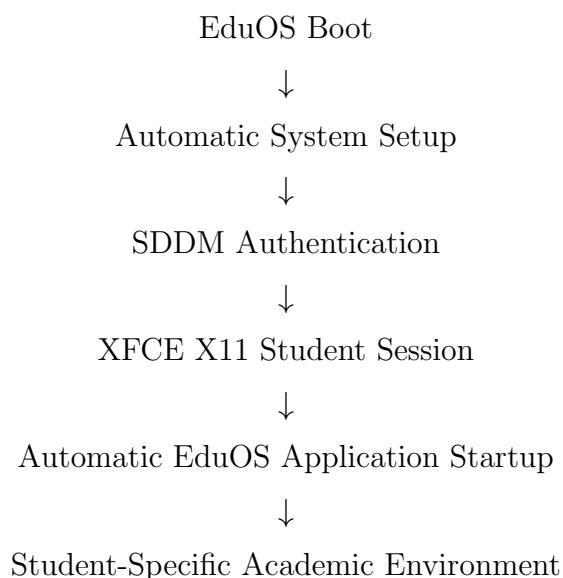
The complete proposed architecture can be divided into four major layers:

- **EduOS Client Layer:** The Linux operating environment used by students on laboratory machines.
- **Management Layer:** Responsible for users, machines, configuration profiles, and centralized administration.
- **EduLab Layer:** Responsible for academic tasks, submissions, evaluation, and results.
- **Infrastructure Layer:** Provides supporting services such as databases, virtualization, containers, and deployment infrastructure.

At the current prototype stage, most implemented functionality is within the EduOS Client Layer.

2.1.1 High-Level Design

The current prototype follows the workflow:



The student's Linux account acts as the primary identity.

After successful authentication, the EduOS application obtains the currently logged-in username directly from the operating system. Therefore, a second application-level login is not required.

The username is currently mapped to academic information stored in a local structured JSON file. In later stages, this local data source can be replaced by a centralized API and database.

2.1.2 Technical Stack

The technologies currently used in the prototype are shown in Table 2.1.

Technology	Purpose
Arch Linux	Base operating system used by the current prototype.
ArchISO	Creation of the customized bootable EduOS image.
XFCE	Desktop environment used for graphical student and administrator sessions.
SDDM	Graphical display manager and Linux authentication interface.
Xorg / X11	Graphical display/session environment.
Python	Implementation language for the student-side EduOS application.
Tkinter	Graphical toolkit used for the current student interface.
JSON	Temporary structured storage for student and task information.
systemd	Boot-time execution and service management.
QEMU/KVM	Virtualization environment used to test generated ISO images.
Git	Version control for source code and ArchISO configuration.
FastAPI	Planned technology for the centralized management API.
PostgreSQL	Planned persistent data storage for future backend services.
Podman	Planned isolation mechanism for automated evaluation workloads.

Table 2.1: EduOS Technical Stack

2.2 Implementation Details

2.2.1 Module 1: Arch Linux ISO Setup

The first implementation module focuses on creating the base EduOS operating-system environment using Arch Linux and ArchISO.

The purpose of this module is to produce a bootable customized Linux image containing the graphical environment, runtime dependencies, system services, and configuration required by EduOS.

The ArchISO profile is maintained under:

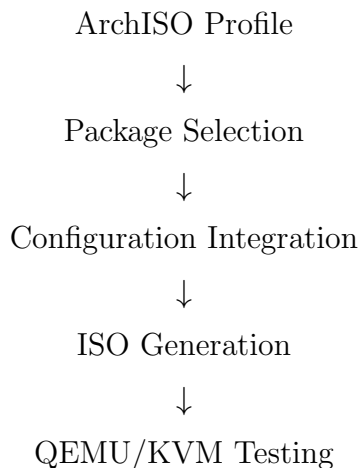
`code/archiso/profile`

The current image contains:

- Arch Linux base system.
- XFCE desktop environment.
- SDDM display manager.
- Xorg graphical environment.
- Mesa graphics support.
- Python runtime.
- Tkinter.
- systemd.
- Networking and basic system administration utilities.

The ISO is generated using the `mkarchiso` utility.

The general module workflow is:



QEMU/KVM allows the complete EduOS environment to be tested in an isolated virtual machine without modifying the host operating system.

During implementation, an important issue occurred in the graphical session startup. SDDM authentication succeeded, but the XFCE session terminated because the Wayland XFCE session was being selected.

The problem was resolved by explicitly configuring SDDM to use the X11 graphical environment.

The relevant configuration is stored at:

```
/etc/sddm.conf.d/10-eduos-x11.conf
```

After this configuration was introduced, SDDM could authenticate users and successfully launch the XFCE X11 session.

This module therefore provides the bootable and reproducible operating-system foundation for EduOS.

2.2.2 Module 2: User Base and Student Environment Setup

The second module focuses on establishing a multi-user academic environment on top of the EduOS operating system.

Instead of representing students only inside an application, students are implemented as genuine Linux user accounts.

The current prototype contains five test student accounts:

- `student1000`
- `student1001`
- `student1002`
- `student1003`
- `student1004`

A separate `eduos` account is retained for system administration, maintenance, and debugging.

Each student receives:

- A separate Linux username.
- A unique Linux user identity.
- An independent home directory.
- A separate login session.

- A configured password.
- An associated roll number.
- Student-specific academic information.

Student account creation is automated using:

```
/usr/local/bin/eduos-setup
```

The setup script is executed by a systemd service during system startup.

The service runs before the graphical display manager, ensuring that the required student accounts exist before SDDM presents the login screen.

The provisioning process is designed to be idempotent. Re-running the setup process maintains the required user configuration without creating duplicate student accounts.

The student interface is implemented in Python using Tkinter:

```
/usr/local/bin/eduos-student-app
```

After login, the application retrieves the currently authenticated Linux username. Student academic information is currently read from:

```
/usr/share/eduos/student-data.json
```

The interface displays information including:

- Student name.
- Roll number.
- Linux username.
- Assigned academic tasks.
- Task status.
- Due information.

Different test students are assigned different tasks to demonstrate that the environment is personalized according to the authenticated identity.

The application launches automatically after student login and opens in fullscreen mode.

The administrative `eduos` account is excluded from this student workflow and instead receives access to the normal XFCE desktop.

2.3 System Diagrams

2.3.1 Use Case Diagram

The use case diagram represents the major interactions between students, administrators, and the EduOS system.

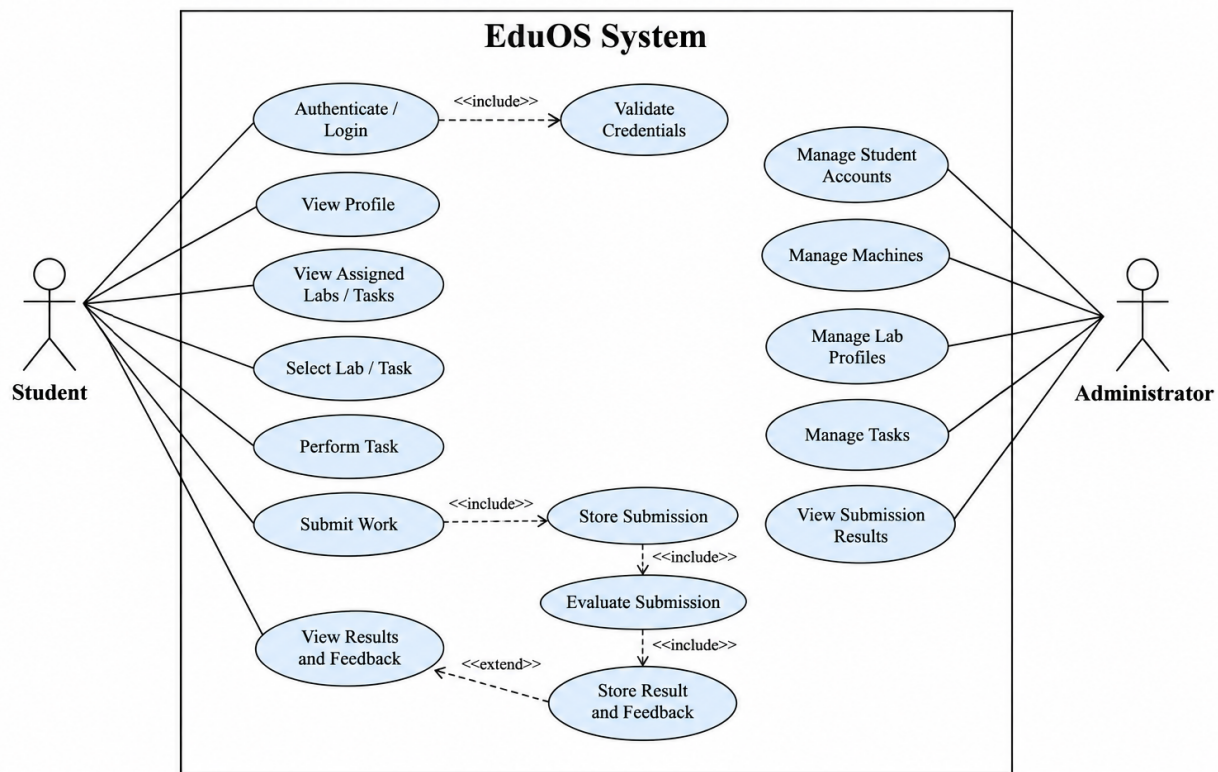


Figure 2.1: EduOS Use Case Diagram

2.3.2 Use Case Specifications

The use case specifications describe important system interactions in greater detail by defining actors, goals, preconditions, normal flows, alternative flows, and postconditions.

EduOS – Use Case Specifications

Use Case Specification 1: Student Login		Use Case Specification 2: View Assigned Tasks	
Use Case ID	UC-01	Use Case ID	UC-02
Use Case Name	Student Login	Use Case Name	View Assigned Tasks
Primary Actor	Student	Primary Actor	Student
Goal	Authenticate the student and provide access to the EduOS student environment.	Goal	Allow the logged-in student to view academic tasks assigned to their account.
Preconditions	A valid student Linux account exists in the system.	Preconditions	Student is successfully authenticated and the EduOS student application is running.
Trigger	The student selects their account and enters the password on the SDDM login screen.	Trigger	The EduOS student application starts after login.
Main Flow	1. Student selects/enters the username. 2. Student enters the password. 3. SDDM validates the credentials. 4. XFCE X11 session starts. 5. EduOS student application launches automatically. 6. Student-specific information is loaded.	Main Flow	1. Application determines the currently logged-in Linux username. 2. Student data is retrieved from the local data store. 3. Tasks assigned to the student are identified. 4. Task title, status, and due information are displayed.
Postconditions	The student is authenticated and enters the personalized EduOS academic environment.	Postconditions	The student can view the tasks assigned to their account.
Alternative Flow	If incorrect credentials are entered, authentication fails and the student can retry.	Alternative Flow	If no tasks are assigned, the system displays that no tasks are currently available.
Exceptions	Student account does not exist, authentication service fails, or XFCE session fails to start.	Exceptions	Student data is missing, malformed, or cannot be loaded.

Use Case Specification 3: Manage Student Users		Use Case Specification 4: Access Personalized Student Environment	
Use Case ID	UC-03	Use Case ID	UC-04
Use Case Name	Manage Student Users	Use Case Name	Access Personalized Student Environment
Primary Actor	Administrator	Primary Actor	Student
Goal	Create, inspect, and maintain student Linux accounts used by EduOS.	Goal	Provide the student with an EduOS interface personalized according to their authenticated identity.
Preconditions	Administrator has access to the EduOS administrative account and sufficient system privileges.	Preconditions	Student has successfully logged into EduOS.
Trigger	Administrator performs user-management operations or the EduOS provisioning service runs during boot.	Trigger	XFCE starts the EduOS student application through autostart.
Main Flow	1. Existing student accounts are checked. 2. Missing student accounts are created. 3. Passwords and account information are configured. 4. Separate home directories are assigned. 5. User information is verified.	Main Flow	1. EduOS application starts automatically. 2. Current Linux username is detected. 3. Corresponding student record is loaded. 4. Student name, roll number, and tasks are displayed. 5. Application enters fullscreen mode.
Postconditions	Required student accounts exist and are correctly configured for authentication.	Postconditions	The student receives a personalized academic workspace.
Alternative Flow	If a student account already exists, its required configuration is updated instead of creating a duplicate account.	Alternative Flow	If the logged-in user is the "eduos" administrator account, the student interface does not start and normal XFCE remains available.
Exceptions	Account creation fails, insufficient privileges are available, or required system commands fail.	Exceptions	Student record cannot be found or the application fails to load the data source.

Figure 2.2: EduOS Use Case Specifications

2.3.3 Class Diagram

The class diagram represents the major conceptual entities of EduOS and the relationships between users, students, administrators, tasks, submissions, results, machines, and supporting system components.

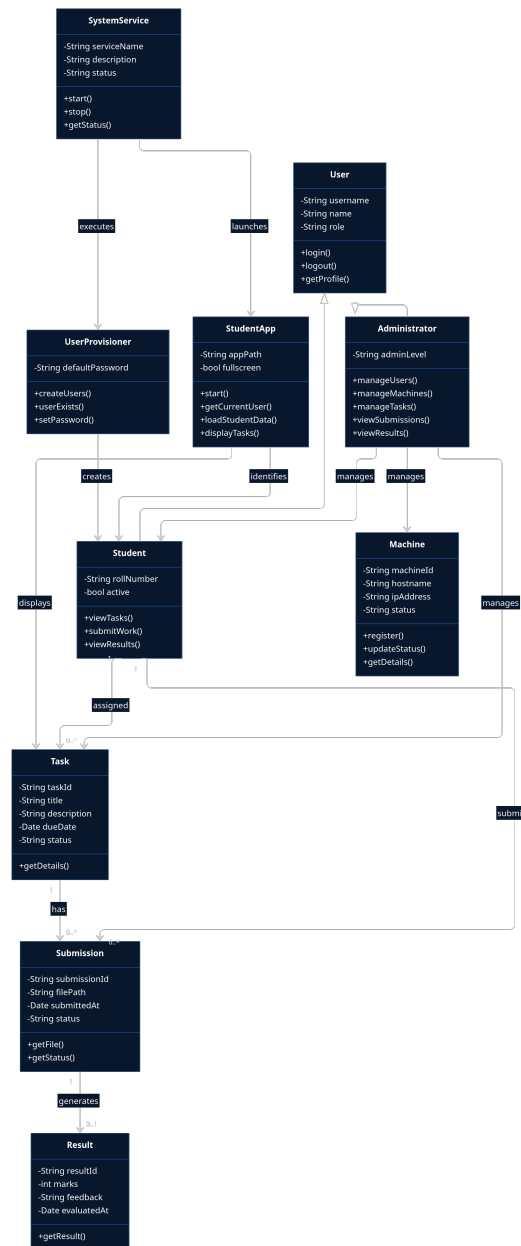


Figure 2.3: EduOS Class Diagram

2.3.4 Activity Diagram

The activity diagram represents the workflow of the EduOS system. Swimlanes are used to distinguish the responsibilities of the student and the major EduOS components.

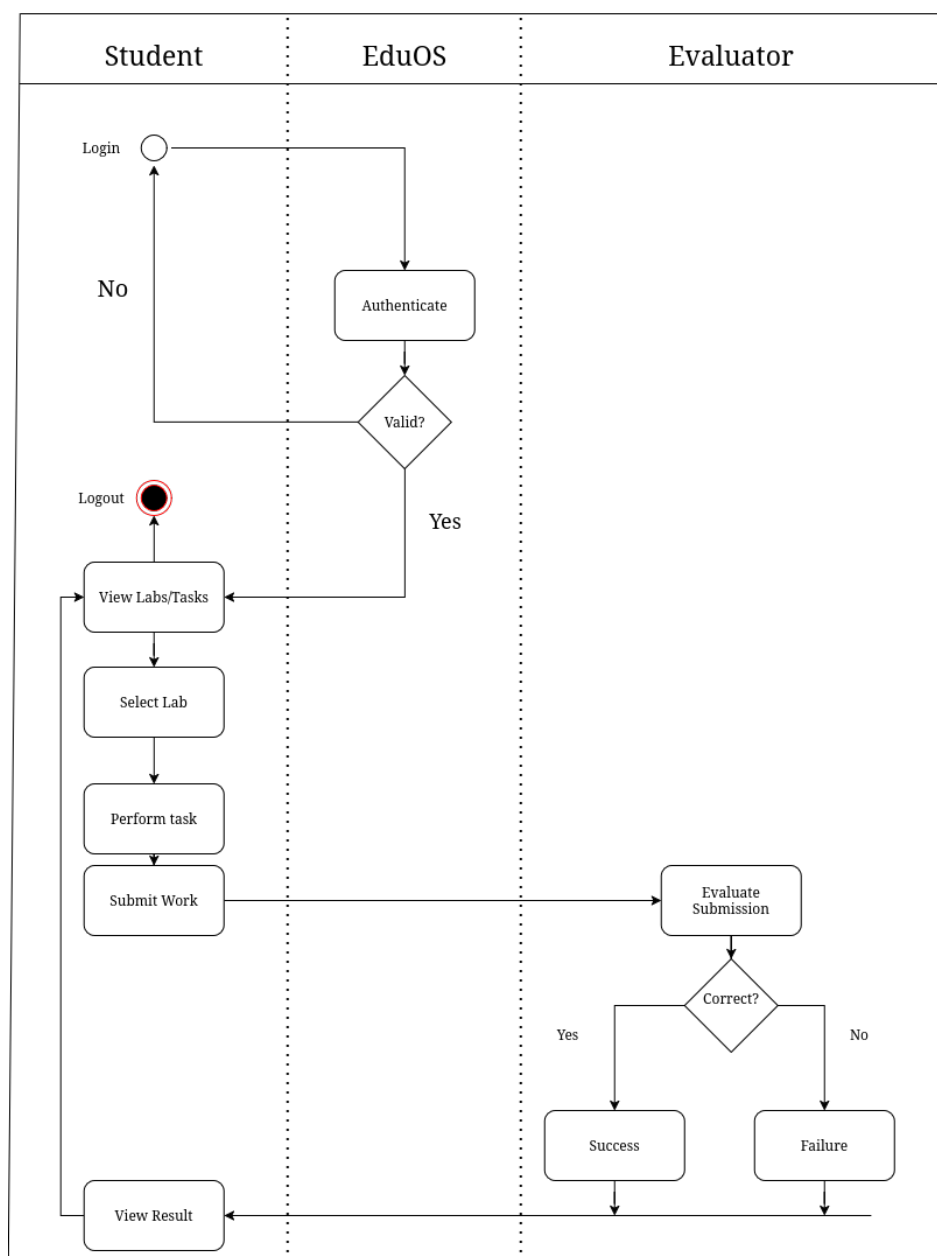


Figure 2.4: EduOS Activity / Swimlane Diagram

2.3.5 Data Flow Diagrams

Data Flow Diagrams are used to represent how information moves between users, EduOS processes, and data stores.

Level 0 Data Flow Diagram

The Level 0 DFD provides a context-level representation of EduOS and its interactions with external entities.

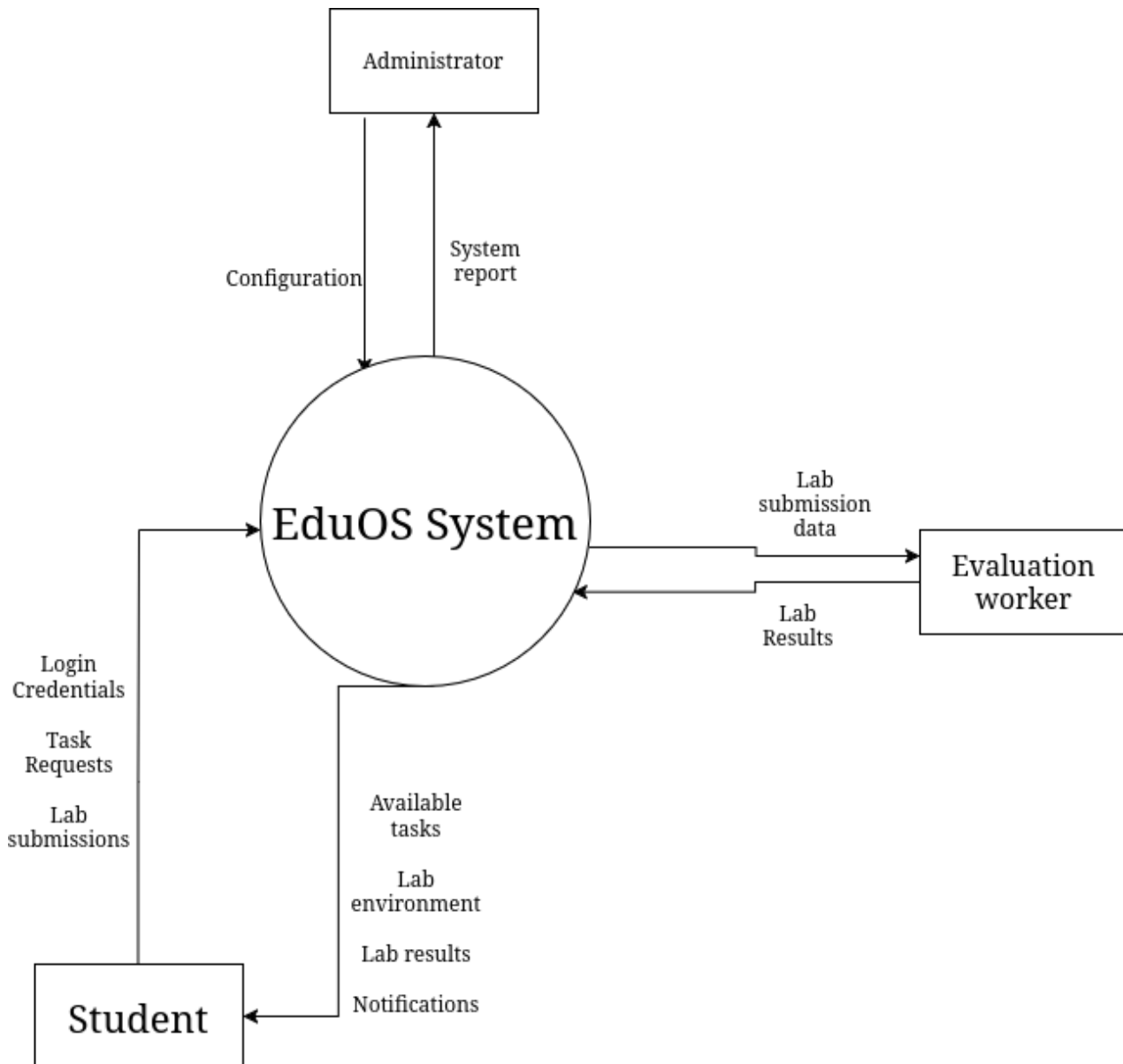


Figure 2.5: EduOS Level 0 Data Flow Diagram

Level 1 Data Flow Diagram

The Level 1 DFD decomposes EduOS into its major functional processes, including authentication, machine management, task management, evaluation, and results management.

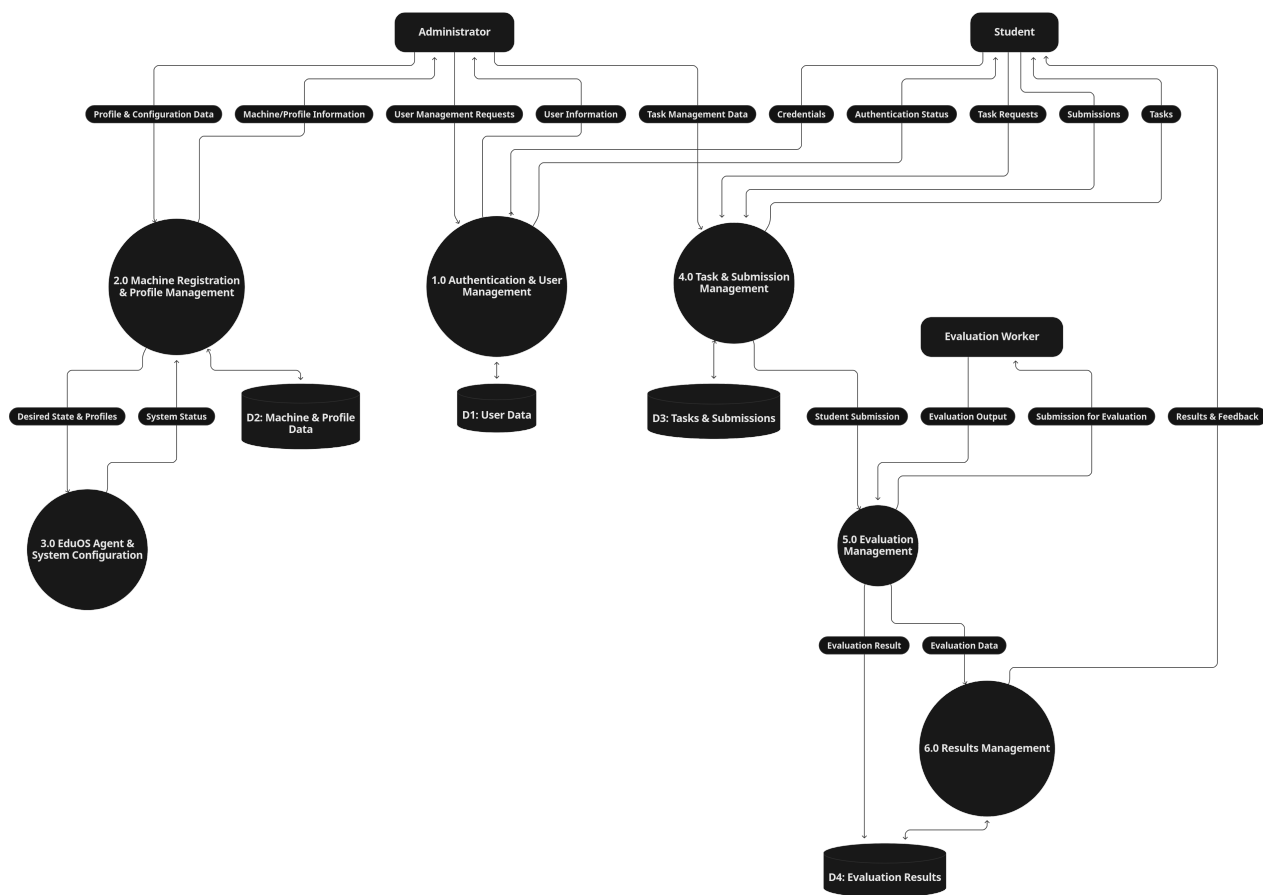


Figure 2.6: EduOS Level 1 Data Flow Diagram

Level 2 Data Flow Diagram

The Level 2 DFD provides a more detailed decomposition of the task and submission workflow.

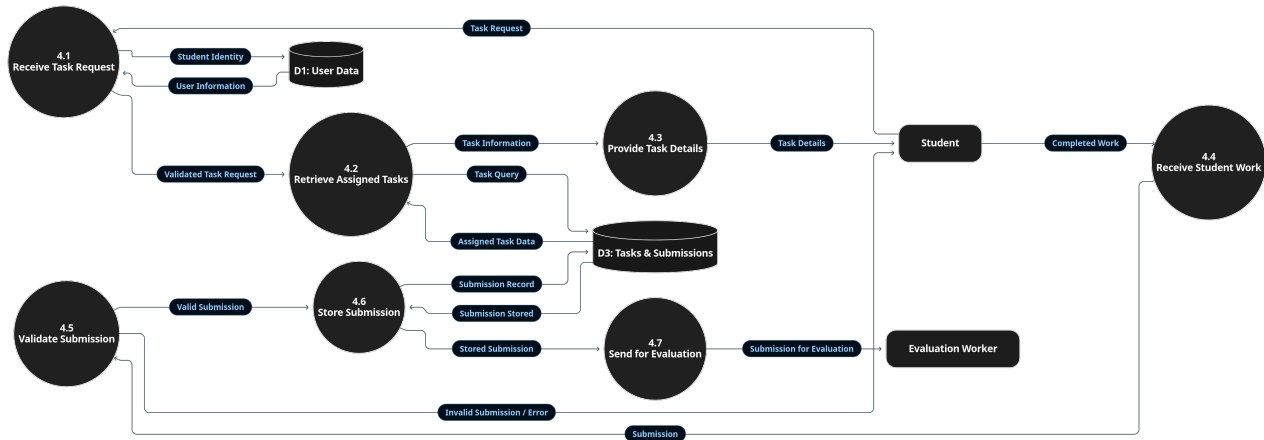


Figure 2.7: EduOS Level 2 Data Flow Diagram

2.3.6 Sequence Diagram

The sequence diagram represents the chronological exchange of messages between the student and major EduOS components.

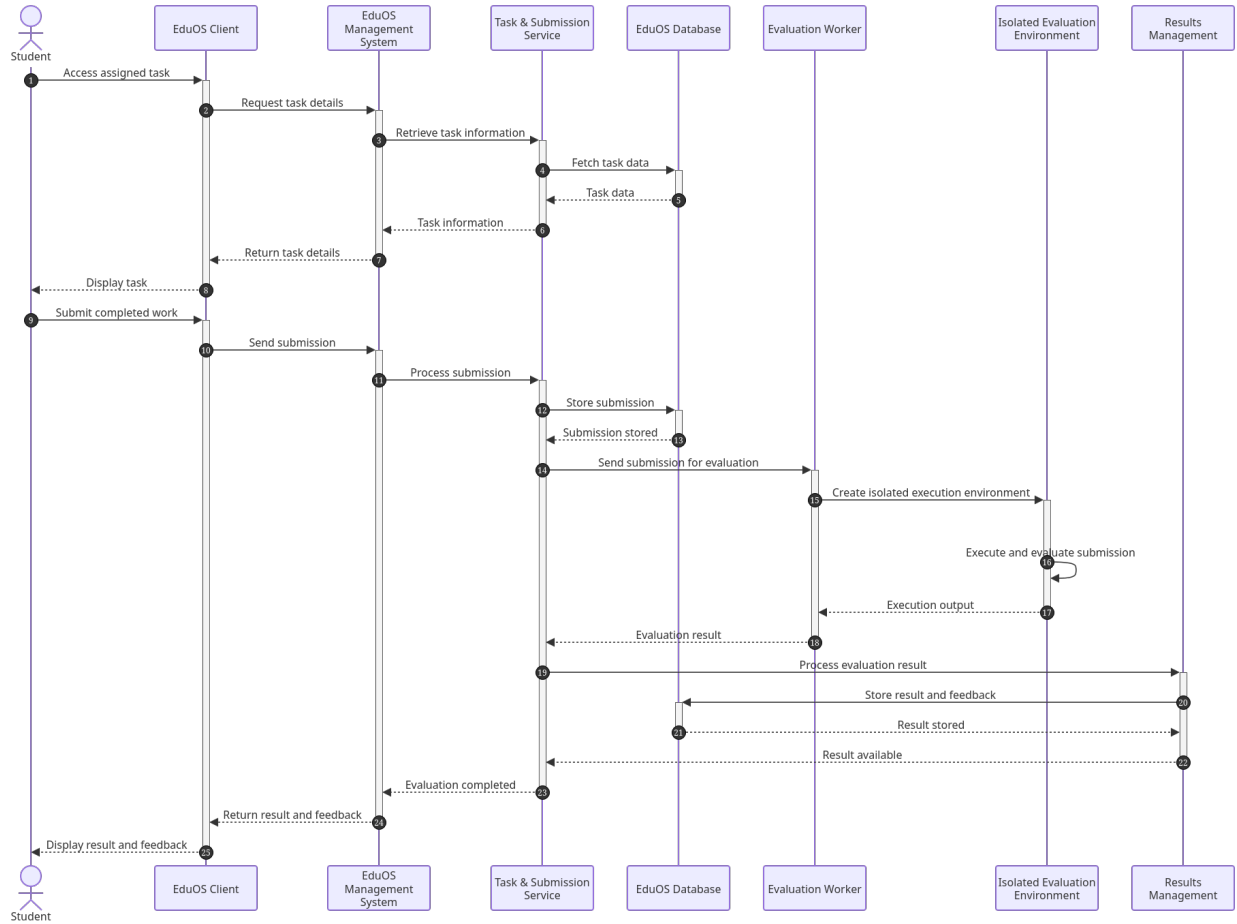


Figure 2.8: EduOS Sequence Diagram

2.3.7 Collaboration Diagram

The collaboration diagram represents the structural interaction between major EduOS components and emphasizes their communication relationships.

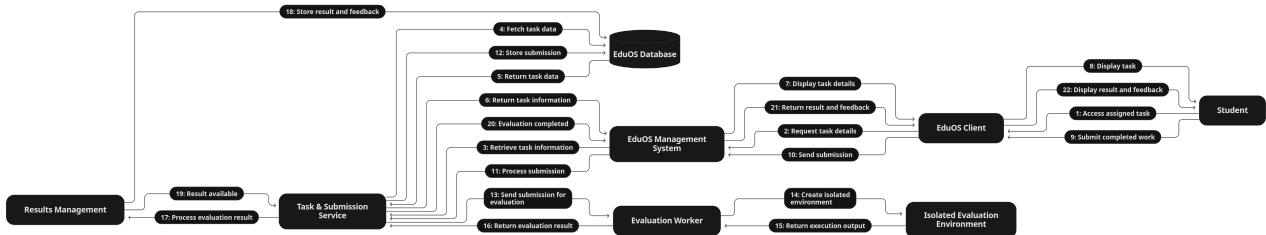


Figure 2.9: EduOS Collaboration Diagram

2.3.8 Gantt Chart

The Gantt chart represents the development schedule and planned sequence of major EduOS activities.

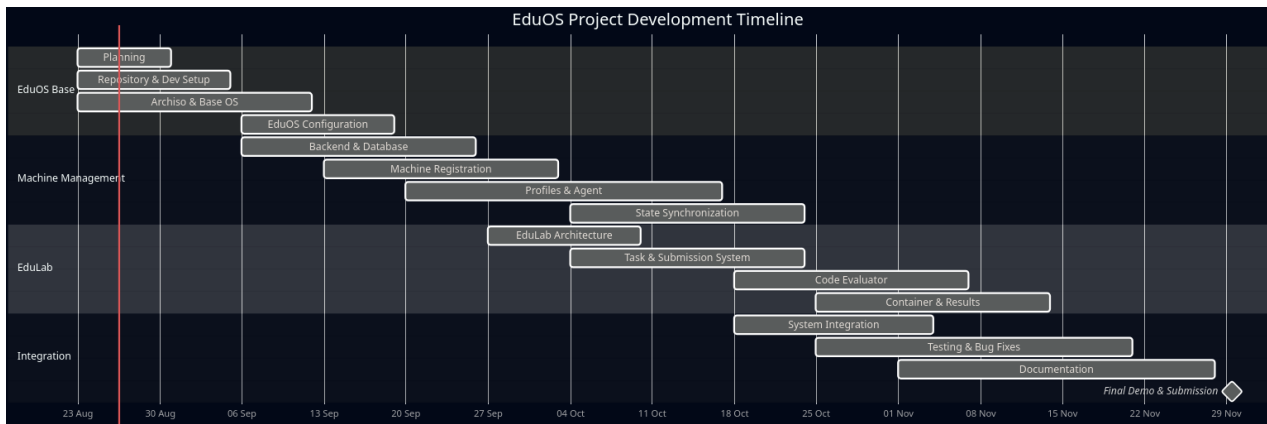


Figure 2.10: EduOS Project Development Timeline

Results and Evaluation

3.1 Testing Methodology

The current EduOS prototype was evaluated primarily through functional and integration testing.

Since the project is currently at the proof-of-concept stage, the main objective of testing was to verify whether the operating-system image, authentication process, student sessions, automated setup, and student application operate correctly as a complete workflow.

The following testing approaches were used:

- **Component Testing:** Individual scripts and configuration files were checked before being included in the ISO. This included the account provisioning script, JSON student data, application startup configuration, and Python student application.
- **Integration Testing:** The interaction between systemd, SDDM, XFCE, Linux users, and the EduOS student application was tested after generating the complete system image.
- **Virtual Machine Testing:** Generated ISO images were booted using QEMU/KVM to test the system from a clean startup environment.
- **Multi-User Testing:** Different student accounts were authenticated to confirm that users receive separate Linux identities, home directories, and student-specific academic information.
- **Administrative Testing:** The `eduos` account was tested separately to verify that administrative access to the normal XFCE environment remains available.

3.2 Performance Metrics

At the current prototype stage, evaluation focuses primarily on functional correctness rather than large-scale performance benchmarking.

Table 3.1 summarizes the major prototype validation results.

Test	Expected	Result
Custom EduOS ISO boots in QEMU/KVM	Successful boot	Passed
SDDM graphical login is displayed	Login available	Passed
Student accounts authenticate	Valid login	Passed
Five separate student Linux accounts exist	5 users	Passed
Separate student home directories are created	Independent homes	Passed
XFCE X11 session starts after authentication	Desktop session	Passed
EduOS student application starts automatically	Automatic startup	Passed
Application detects authenticated Linux user	Correct identity	Passed
Different users receive different task information	Personalization	Passed
Student application opens in fullscreen mode	Fullscreen interface	Passed
Administrative account retains normal XFCE desktop	Admin access	Passed

Table 3.1: Prototype Functional Evaluation Results

3.3 Analysis

The prototype successfully demonstrates the primary client-side concept of EduOS.

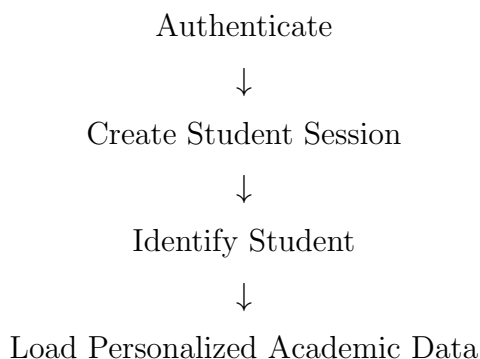
A customized Linux image can be generated and booted inside a virtual machine. Student accounts are created at the operating-system level, allowing standard Linux authentication and session isolation to be used instead of implementing a separate simulated authentication system.

The system also demonstrates identity-based personalization. Once a student has authenticated, the EduOS application can determine the current Linux user and retrieve the corresponding academic data.

This demonstrates a complete prototype flow:

Boot

↓



Several implementation challenges were encountered during development.

The most significant graphical-session issue occurred when the display manager attempted to launch XFCE using Wayland. The problem was diagnosed through session behavior and configuration inspection, and the system was configured to use X11 instead.

Another important challenge was ensuring that the student accounts were available before graphical authentication. This was addressed by integrating account provisioning with systemd and controlling the startup order relative to SDDM.

The current results therefore demonstrate that the underlying EduOS client architecture is technically feasible.

However, the current academic data is still stored locally and the prototype does not yet provide the centralized management, submission, or automated evaluation components planned for the full system.

Conclusion

4.1 Summary of Work

The mid-semester EduOS prototype establishes the foundation of a Linux-based academic computing environment.

Two major implementation modules were completed during the current stage.

The first module produced a customized bootable Arch Linux image using ArchISO. The image integrates XFCE, SDDM, Xorg, Python, Tkinter, systemd, and the supporting packages required by the current prototype.

The second module implemented a multi-user student environment. Multiple real Linux student accounts are automatically provisioned, each with an independent identity, home directory, login session, and associated academic information.

A student-side Python/Tkinter application was also integrated into the operating system. It automatically identifies the currently authenticated Linux user and displays student-specific information and assigned tasks.

Together, these modules demonstrate the central client-side concept of EduOS: a student can boot into a standardized academic operating environment, authenticate using a Linux account, and automatically receive a personalized workspace.

4.2 Key Findings

The major findings from the current implementation are:

- **Linux-native authentication can be reused for academic identity.** The student application does not require a second login because it can use the identity of the authenticated Linux session.
- **Student environments can be provisioned automatically.** systemd-based initialization allows user accounts and required configuration to be established during system startup.
- **A customized academic Linux image can be reproduced using ArchISO.** Package selection and system configuration are maintained within the project profile rather than being manually applied after installation.
- **Student-specific interfaces can coexist with an administrative environment.** Student accounts automatically enter the EduOS interface, while the administrative account retains access to normal XFCE.

- **Virtual-machine testing is effective during operating-system development.** QEMU/KVM allowed repeated ISO builds and boot testing without modifying the host machine.

4.3 Future Work and Recommendations

The next development stage will extend the current client-side prototype into the broader EduOS architecture.

Planned work includes:

1. Develop a centralized management API using FastAPI.
2. Replace the local JSON student data with persistent database-backed information.
3. Implement centralized student and machine management.
4. Introduce configurable laboratory profiles for different courses and practical sessions.
5. Implement task distribution and submission handling.
6. Develop the first automated programming evaluator.
7. Introduce isolated execution using containers.
8. Store and present evaluation results and feedback.
9. Connect EduOS clients to the management server so that laboratory machines can receive centrally controlled configurations.

4.4 Final Remarks

The current prototype demonstrates that EduOS can provide more than a conventional customized Linux desktop.

By combining a reproducible operating-system image, native Linux user management, automated provisioning, and a student-specific interface, the prototype establishes a foundation for a managed academic computing platform.

The next stages of development will focus on moving from the current local proof of concept toward centralized management and automated academic assessment.